Practical – Week7

1. Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

```c
#include <stdio.h>
#include <stdlib.h>

int global_var = 10;
static int static_var = 20;

void code_function() {
    printf("Code address   : %p\n", (void *)code_function);
}

int main() {
    int local_var = 30;
    static int static_local = 40;

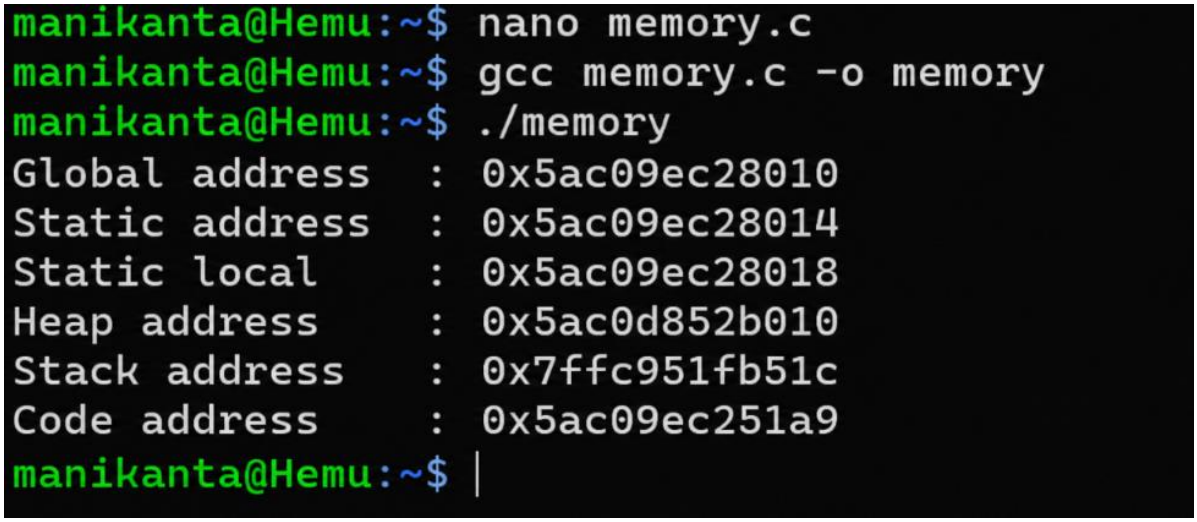
    int *heap_var = malloc(sizeof(int));
    *heap_var = 50;

    printf("Global address : %p\n", (void *)&global_var);
    printf("Static address : %p\n", (void *)&static_var);
    printf("Static local   : %p\n", (void *)&static_local);
    printf("Heap address   : %p\n", (void *)heap_var);
    printf("Stack address  : %p\n", (void *)&local_var);

    code_function();

    free(heap_var);

    return 0;
}
```



Report: The C program successfully prints the addresses of code, global, static, heap, and stack variables. The experiment demonstrates the Linux process address space and how different memory segments store and manage data during program execution.

2. Make use of the /proc//maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 10;

int main() {
    int local_var = 20;
    int *heap_var = malloc(sizeof(int));

    *heap_var = 30;

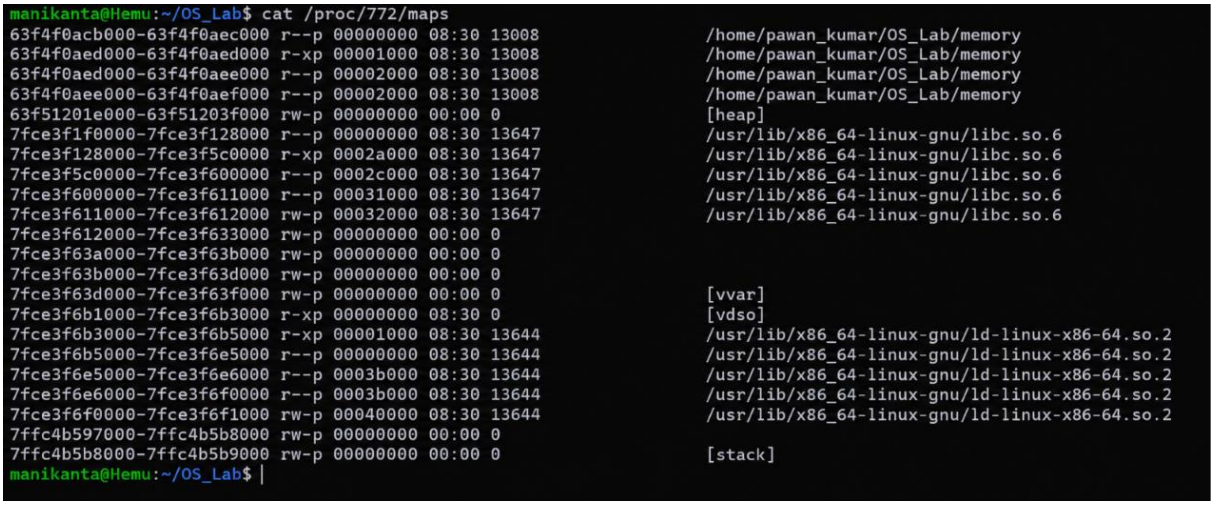
    printf("Process ID: %d\n", getpid());
    printf("Global address: %p\n", (void *)&global_var);
    printf("Stack address: %p\n", (void *)&local_var);
    printf("Heap address: %p\n", (void *)heap_var);

    printf("Process is running...\n");

    while (1) {
        sleep(10);
    }

    free(heap_var);
    return 0;
}
```

```
manikanta@Hemu:~$ cd OS_Lab
-bash: cd: OS_Lab: No such file or directory
manikanta@Hemu:~$ mkdir OS_Lab
manikanta@Hemu:~$ nano memory.c
manikanta@Hemu:~$ gcc memory.c -o memory
manikanta@Hemu:~$ ./memory &
[1] 772
Process ID: 772
Global address: 0x63f4f0aef010
Stack address: 0x7ffe4b5b673c
Heap address: 0x63f51201e010
Process is running...
manikanta@Hemu:~$
```



Report: The /proc/[PID]/maps file helps us understand how Linux organizes a process's virtual memory. Memory analysis tools such as pmap, ps, and top help inspect memory usage and mappings of a running process.